REPÚBLICA BOLIVARIANA DE VENEZUELA
MINISTERIO DEL PODER POPULAR PARA
LA EDUCACIÓN UNIVERSITARIA
INSTITUTO UNIVERSITARIO DE TECNOLOGÍA
"ELÍAS CALIXTO POMPA"
INFORMÁTICA
ANÁLISIS Y DISEÑO DE SISTEMAS II

SISTEMA DE GESTIÓN DE ASISTENCIA MEDIANTE CÓDIGOS QR PARA EL IUTECP

Informe Final

Facilitador: William Hernández

Integrantes:

Fernando Rodríguez C.I: 31.853.086

Daniel Dias C.I: 30.937.908

Guatire, 03 de Septiembre de 2026

Índice general

RESUMEN IV

1 INTRODUCCIÓN 1

2 PROPÓSITO Y ALCANCE DEL SISTEMA 2

2.1 Objetivo General 2

2.2 Objetivos Específicos 2

2.3 Alcance 3

3 ARQUITECTURA GENERAL 4

4 FUNCIONALIDADES POR ROL 5

4.1 Administrador 5

4.2 Profesor 5

4.3 Estudiante 6

5 MODELO DE CASOS DE USO 7

5.1 Actores del Sistema 7

5.2 Diagrama de Casos de Uso 7

5.3 Especificaciones de Casos de Uso 9

6 METODOLOGÍA DE DESARROLLO: SCRUM ADAPTADO 15

6.1 Fundamentación Teórica 15

6.2 Tabla Metodológica Adaptada 16

7 ESPECIFICACIÓN DE LOS SPRINTS 18

7.1 Sprint 1 – Setup y Autenticación (Semanas 1–2) 18

7.1.1 Actividades realizadas 18

7.1.2 Producto resultante 18

7.2 Sprint 2 – Núcleo QR: Generar y Escanear (Semanas 3–4) 19
7.2.1 Actividades realizadas 19
7.2.2 Producto resultante 19
7.3 Sprint 3 – Panel Admin: Gestión y Reportes (Semanas 5–6) 19
7.3.1 Actividades realizadas 19
7.3.2 Producto resultante 20
7.4 Sprint 4 – Pruebas, Validación y Entrega (Semanas 7–8) 20
7.4.1 Actividades realizadas 20
7.4.2 Producto resultante 20

8 MODELO DE PRUEBAS DEL PROGRAMA 21
8.1 Estrategia de Pruebas 21
8.2 Plan por Módulos Reales 22
8.3 Descripción del Modelo de Pruebas 22

9 FACTIBILIDAD ECONÓMICA 24
9.1 Costos Humanos 24
9.2 Costos Tecnológicos 24
9.3 Costos Totales del Proyecto 24
9.4 Ahorros y Beneficios 25

10 DISEÑO DE LA BASE DE DATOS 26
10.1 Tablas del Sistema 26
10.2 Esquema Relacional 27

11 CONSTRUCCIÓN DEL SISTEMA 29
11.1 Herramientas de Programación 29
11.2 Librerías de Programas 29
11.3 Fragmento de Codificación 30

12 INTERFACES DEL SISTEMA 33

12.1 Interfaces del Profesor 33
12.2 Interfaces del Estudiante 33
12.3 Interfaces del Administrador 33
12.4 Galería de Interfaces 35

13 DISEÑO NO FUNCIONAL **38**
13.1 Seguridad y Privacidad 38
13.1.1 Autenticación y Control de Acceso 38
13.1.2 Protección contra Ataques 38
13.1.3 Auditoría 39
13.2 Usabilidad 39
13.3 Accesibilidad 39

CONCLUSIÓN **40**

REFERENCIAS **41**

RESUMEN

El presente informe final documenta de manera integral el desarrollo del Sistema de Gestión de Asistencia mediante Códigos QR para el Instituto Universitario de Tecnología “Elías Calixto Pompa” (IUTECP). El sistema automatiza el registro de asistencia de los estudiantes mediante la generación de códigos QR por parte del profesor y su posterior escaneo con dispositivos móviles, eliminando las limitaciones del método tradicional basado en listas impresas. El documento abarca la introducción y justificación del proyecto, el propósito y alcance con sus objetivos generales y específicos, la arquitectura tecnológica, las funcionalidades por rol, el modelo de casos de uso, la metodología de desarrollo Scrum adaptada con su tabla metodológica por sprints y su especificación detallada, el modelo de pruebas del programa, la factibilidad económica, el diseño de la base de datos, la construcción del sistema, sus interfaces y el diseño no funcional.

CAPÍTULO I

INTRODUCCIÓN

El control de asistencia en instituciones educativas constituye un proceso fundamental para la gestión académica. Tradicionalmente, este proceso se realiza mediante listas impresas que el profesor debe portar y marcar manualmente durante cada clase, lo que consume tiempo valioso, implica un gasto recurrente de papel e impresión, y está sujeto a errores humanos, suplantación de identidad y pérdida de registros.

En el contexto del Instituto Universitario de Tecnología "Elías Calixto Pompa" (IUTECP), se hace necesario contar con una solución tecnológica que optimice este proceso, reduzca los costos asociados a la impresión de planillas y ofrezca información confiable y oportuna sobre el ausentismo estudiantil.

Ante esta problemática, el presente proyecto plantea el desarrollo de un **Sistema de Gestión de Asistencia mediante Códigos QR**, una aplicación web cliente-servidor que permite al profesor generar, durante cada sesión de clase, un código QR único y temporal que el estudiante escanea con su dispositivo móvil para registrar su asistencia de forma automática y segura.

El sistema se fundamenta en la capacidad de los códigos QR para codificar información de forma rápida y legible por dispositivos con cámara, y en el uso de tecnologías web estándar ampliamente disponibles, lo que garantiza compatibilidad con los equipos y dispositivos existentes en la institución sin necesidad de inversiones adicionales.

A continuación se desarrollan de forma ordenada todos los aspectos del proyecto: su propósito y alcance, la arquitectura, las funcionalidades, el modelo de casos de uso, la metodología de desarrollo, el modelo de pruebas, la factibilidad económica, el diseño de la base de datos, la construcción del sistema, sus interfaces y su diseño no funcional.

CAPÍTULO II

PROPÓSITO Y ALCANCE DEL SISTEMA

2.1. Objetivo General

Desarrollar un Sistema de Gestión de Asistencia mediante Códigos QR para el Instituto Universitario de Tecnología "Elías Calixto Pompa" (IUTECP), que automatice el registro de asistencia de los estudiantes a través de la generación y el escaneo de códigos QR, garantizando rapidez, seguridad y confiabilidad en los datos.

2.2. Objetivos Específicos

- Identificar el proceso actual de toma de asistencia y las debilidades del método tradicional basado en listas impresas.
- Definir y modelar los requerimientos funcionales del sistema mediante casos de uso, identificando los actores administrador, profesor y estudiante.
- Diseñar la base de datos relacional que almacene usuarios, cursos, inscripciones, asistencias, sesiones QR y auditoría.
- Desarrollar una aplicación web con tecnologías estándar (PHP, MariaDB, HTML5, CSS3 y JavaScript) que implemente las funcionalidades definidas.
- Elaborar un modelo de pruebas funcionales que valide el comportamiento del sistema por módulo y de forma integral.
- Documentar la factibilidad económica del proyecto considerando la infraestructura existente y los ahorros generados.

2.3. Alcance

El presente proyecto abarca el análisis, el diseño, el desarrollo y las pruebas de la aplicación web de gestión de asistencia mediante códigos QR. El sistema contempla tres roles de usuario —administrador, profesor y estudiante— con funcionalidades que van desde la gestión académica hasta la consulta de reportes de ausentismo.

La aplicación se concibe como una solución web accesible desde cualquier dispositivo con navegador y cámara, sin requerir la instalación de aplicaciones de terceros. El sistema se instalará sobre la infraestructura de servidores y equipos ya existentes en la universidad, utilizando tecnologías de código abierto para evitar costos de licenciamiento.

Queda fuera del alcance la implementación física masiva del sistema en todas las dependencias del instituto durante la fase de desarrollo del proyecto, así como la elaboración de un módulo de calificaciones académicas más allá de la estructura de datos contemplada.

CAPÍTULO III

ARQUITECTURA GENERAL

El sistema se concibe bajo una arquitectura **cliente-servidor** utilizando tecnologías web estándar. El *backend* se desarrolla en PHP 8.4 con conexiones seguras a una base de datos MariaDB mediante sentencias preparadas PDO, mientras que el *frontend* emplea HTML5, CSS3 y JavaScript vanilla para garantizar compatibilidad y rendimiento sin dependencias externas.

El entorno de desarrollo se gestiona mediante `DDEV`, una herramienta basada en Docker que permite ejecutar el sistema localmente con PHP 8.4, MariaDB y servidor nginx, replicando fielmente el entorno en el que se desplegará sobre los equipos de la institución.

Las ventajas de esta arquitectura son:

- Multiplataforma: funciona en cualquier navegador moderno y en cualquier sistema operativo.
- Compatibilidad móvil: los estudiantes pueden escanear el QR desde cualquier teléfono inteligente con cámara.
- Código abierto: no requiere compra de licencias de software.
- Seguridad: uso de sentencias preparadas PDO y sanitización de entradas.

CAPÍTULO IV

FUNCIONALIDADES POR ROL

El sistema ofrece funcionalidades diferenciadas según el rol del usuario autenticado.

4.1. Administrador

El administrador contará con un panel integral de gestión que incluye:

- Gestión completa de usuarios: creación, activación y desactivación de cuentas de profesores y estudiantes.
- Administración de cursos: asignación de profesores a secciones, creación y edición de cursos.
- Gestión de periodos académicos y carga masiva de inscripciones desde archivos CSV.
- Consulta de reportes de asistencia filtrados por curso, semana o estudiante.
- Monitoreo de ausentismo con alertas automáticas para estudiantes en riesgo (25 % a 30 %) y crítico (más de 30 %).
- Visualización de registros de auditoría con los últimos eventos del sistema.
- Generación de respaldos de la base de datos.

4.2. Profesor

El profesor dispondrá de las siguientes capacidades:

- Visualización de los cursos que tiene asignados en el periodo activo.
- Generación de códigos QR únicos para cada sesión de clase, con expiración configurable entre 1 y 60 minutos.

- Rotación automática del código QR cada cierto intervalo de segundos (de 10 a 300), lo que impide la reutilización del código por parte de terceros.
- Consulta en tiempo real del estado de asistencia de sus estudiantes.

4.3. Estudiante

El estudiante podrá:

- Visualizar las materias en las que se encuentra inscrito junto con su progreso en el pensum de la carrera.
- Escanear el código QR proyectado por el profesor utilizando la cámara de su dispositivo móvil.
- Registrar su asistencia de forma automática, siempre que el código QR se encuentre vigente.
- Consultar su historial de asistencia.

CAPÍTULO V

MODELO DE CASOS DE USO

El modelo de casos de uso permite representar gráficamente las interacciones entre los actores externos y las funcionalidades que el sistema debe ofrecer.

5.1. Actores del Sistema

Administrador: Usuario con privilegios totales sobre el sistema. Se encarga de la gestión de usuarios, cursos, periodos académicos, la supervisión general del sistema y la consulta de reportes.

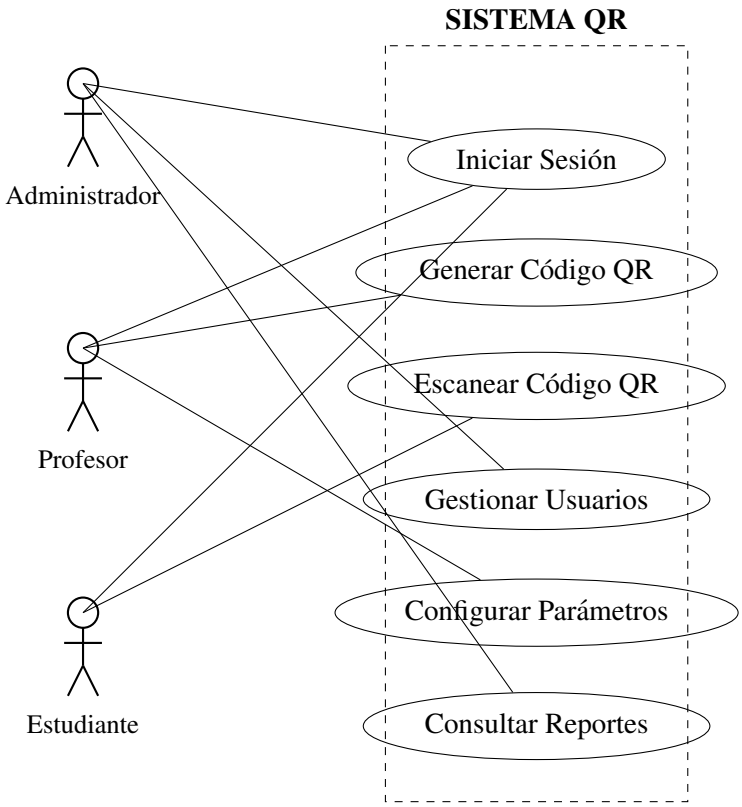
Profesor: Usuario responsable de generar los códigos QR durante sus sesiones de clase, configurar los parámetros de asistencia y consultar el estado de asistencia de sus estudiantes.

Estudiante: Usuario que registra su asistencia mediante el escaneo de códigos QR y consulta su historial de asistencia.

Sistema: Actor secundario que representa los procesos automáticos del sistema, como la validación de expiración del QR, la generación de reportes y el registro de auditoría.

5.2. Diagrama de Casos de Uso

A continuación se presenta la representación gráfica del modelo de casos de uso, elaborada con la notación UML estándar:


SISTEMA QR
Administrador
Profesor
Estudiante
Iniciar Sesión
Generar Código QR
Escanear Código QR
Gestionar Usuarios
Configurar Parámetros
Consultar Reportes

5.3. Especificaciones de Casos de Uso

A continuación se detallan las especificaciones de los casos de uso identificados.

CU-01: Iniciar Sesión	
Actores	Administrador, Profesor, Estudiante
Descripción	Permite a los usuarios autenticarse mediante su cédula o nombre de usuario y contraseña
Precondiciones	El usuario debe poseer una cuenta activa registrada en el sistema
Postcondiciones	El sistema redirige al usuario al panel correspondiente según su rol
Flujo principal	1. El actor ingresa su cédula y contraseña 2. El sistema valida que los campos no estén vacíos 3. El sistema consulta la base de datos y verifica la contraseña 4. El sistema verifica que la cuenta esté activa 5. El sistema inicia la sesión y redirige según el rol
Flujos alternos	**FA-01:** Contraseña incorrecta — el sistema muestra un mensaje de error **FA-02:** Cuenta inactiva — el sistema deniega el acceso **FA-03:** Contraseña temporal — redirige al cambio obligatorio de contraseña

CU-02: Generar Código QR	
Actores	Profesor
Descripción	Permite al profesor generar un código QR único y temporal para una sesión de clase específica
Precondiciones	El profesor debe haber iniciado sesión y tener al menos un curso asignado en el periodo activo
Postcondiciones	Se crea un registro de sesión QR con código único y fecha de expiración
Flujo principal	1. El profesor selecciona el curso y la semana 2. El sistema genera un código único de 32 caracteres hexadecimales criptográficos 3. El sistema calcula la fecha de expiración según la configuración del profesor 4. El sistema presenta el QR en pantalla para que el profesor lo proyecte 5. El sistema inicia la rotación automática según el intervalo configurado
Flujos alternos	**FA-01:** Sin configuración previa — el sistema usa valores por defecto (5 min, 30 s)

CU-03: Escanear Código QR	
Actores	Estudiante
Descripción	Permite al estudiante registrar su asistencia escaneando el código QR generado por el profesor
Precondiciones	El estudiante debe haber iniciado sesión en su dispositivo móvil y estar inscrito en el curso correspondiente
Postcondiciones	Se registra la asistencia del estudiante con estado “presente” para la sesión y semana correspondiente
Flujo principal	1. El estudiante abre el módulo de escaneo desde su dispositivo móvil 2. El sistema activa la cámara del dispositivo para capturar el código QR 3. El estudiante enfoca el QR proyectado por el profesor 4. El sistema decodifica el QR y extrae el código único de sesión 5. El sistema valida que el QR exista y no haya expirado 6. El sistema registra la asistencia como “presente” y muestra confirmación
Flujos alternos	**FA-01:** QR expirado — el sistema rechaza el registro y notifica al estudiante **FA-02:** QR inválido — el sistema muestra un mensaje de error **FA-03:** Estudiante no inscrito — el sistema rechaza el registro

CU-04: Gestionar Usuarios	
Actores	Administrador
Descripción	Permite al administrador crear, modificar, activar o desactivar cuentas de profesores y estudiantes
Precondiciones	El administrador debe haber iniciado sesión
Postcondiciones	Se crean, modifican o actualizan los registros de usuarios según la operación realizada
Flujo principal	1. El administrador accede al módulo de gestión de usuarios 2. El sistema presenta la lista de usuarios con opciones de búsqueda y filtros 3. El administrador selecciona la acción (crear, editar, activar o desactivar) 4. Para crear, ingresa los datos del nuevo usuario (cédula, nombre, correo, rol, carrera) 5. El sistema valida que la cédula no esté duplicada y genera username y contraseña temporal 6. El sistema registra el nuevo usuario en la base de datos
Flujos alternos	**FA-01:** Cédula duplicada — el sistema muestra un mensaje de error **FA-02:** Datos inválidos — el sistema solicita la corrección

CU-05: Configurar Parámetros de Asistencia	
Actores	Profesor
Descripción	Permite al profesor ajustar los parámetros de expiración y rotación del QR
Precondiciones	El profesor debe haber iniciado sesión
Postcondiciones	Se actualizan los valores de configuración del profesor en la base de datos
Flujo principal	1. El profesor accede al módulo de configuración 2. El sistema muestra los valores actuales de expiración y rotación 3. El profesor modifica los parámetros según su preferencia 4. El sistema valida que los valores estén dentro de los rangos permitidos (1–60 min, 10–300 s) 5. El sistema guarda la configuración actualizada
Flujos alternos	**FA-01:** Valores fuera de rango — el sistema indica el rango permitido y solicita corrección

CU-06: Consultar Reportes de Asistencia	
Actores	Administrador
Descripción	Permite al administrador consultar reportes detallados de asistencia filtrados por curso, semana o estudiante
Precondiciones	El administrador debe haber iniciado sesión
Postcondiciones	El sistema muestra los reportes solicitados sin realizar modificaciones en los datos
Flujo principal	1. El administrador accede al módulo de reportes 2. El sistema presenta opciones de filtrado: curso, semana, periodo y estudiante 3. El administrador selecciona los filtros deseados 4. El sistema consulta la base de datos y genera una tabla con resultados 5. El sistema muestra el reporte con asistencias, inasistencias y porcentaje de ausentismo
Flujos alternos	**FA-01:** Sin resultados — el sistema indica que no hay datos disponibles

CAPÍTULO VI

METODOLOGÍA DE DESARROLLO: SCRUM ADAPTADO

6.1. Fundamentación Teórica

Scrum es un marco de trabajo (*framework*) para el desarrollo y mantenimiento de productos de software complejos, definido en la *Scrum Guide* (2020) por Ken Schwaber y Jeff Sutherland. Se basa en el control empírico del proceso mediante tres pilares: transparencia, inspección y adaptación.

Para el presente proyecto, Scrum se adapta a las siguientes restricciones reales:

- **Equipo de dos desarrolladores:** ambos asumen el rol de *Developers* y comparten responsabilidades de *Product Owner* (priorización, clarificación de requisitos) y *Scrum Master* (facilitación de eventos, eliminación de impedimentos) de forma ligera.
- **Duración fija:** ocho semanas distribuidas en cuatro sprints de dos semanas cada uno.
- **Eventos reducidos:**
  - *Sprint Planning* (inicio de cada sprint, 1 hora): definición del Sprint Goal y selección de *Product Backlog Items*.
  - *Daily Scrum* (diario, 10–15 min): sincronización entre los dos desarrolladores.
  - *Sprint Review* (fin de sprint, 30 min): demostración del incremento al *Product Owner*.
  - *Sprint Retrospective* (fin de sprint, 15 min): mejora continua del proceso.
- **Artefactos mínimos:**

- *Product Backlog:* lista priorizada de funcionalidades (tablero físico/Trello).
- *Sprint Backlog:* tareas seleccionadas para el sprint actual.
- *Increment:* código funcional en rama `main` al final de cada sprint.

Esta adaptación mantiene la esencia de Scrum (entregas incrementales, inspección frecuente, adaptación) sin sobrecargar a un equipo de dos personas con ceremonias innecesarias.

6.2. Tabla Metodológica Adaptada

A continuación se presenta la tabla metodológica adaptada al proyecto, donde cada fila corresponde a un sprint, siguiendo el esquema Disciplina / Actividades a Desarrollar / Producto Resultante / Herramientas.

Disciplina	**Actividades a Desarrollar**	**Producto Resultante**	**Herramientas**
Sprint 1 – Setup y Autenticación (Sem 1-2)	1. Configurar entorno DDEV + Git 2. Diseñar y crear esquema BD + seeds 3. Implementar auth: login, roles, sesiones 4. Estructura base MVC ligero	Módulo autenticación funcional + BD poblada	Git, DDEV, PHP 8.4, MariaDB
Sprint 2 – Núcleo QR: Generar y Escanear (Sem 3-4)	1. Profesor: generar QR único, expiración, rotación 2. Estudiante: escanear QR (Camera API), validar, registrar asistencia	Núcleo QR completo (generar + escanear + asistencia)	PHP QR Code, JS Camera API, AJAX fetch
Sprint 3 – Panel Admin: Gestión y Reportes (Sem 5-6)	1. Admin: CRUD usuarios + importación CSV 2. Admin: gestión cursos/inscripciones 3. Admin: reportes asistencia (filtros + CSV export) 4. Backup BD + auditoría	Panel admin completo (gestión + reportes + backup + auditoría)	PHP, MariaDB, CSV, triggers SQL
Sprint 4 – Pruebas, Validación y Entrega (Sem 7-8)	1. Pruebas integrales cross-módulo 2. Corrección bugs detectados 3. Validación móvil (responsive, cámara) 4. Limpieza BD entre ciclos 5. Documentación final + despliegue DDEV	Sistema validado listo para entrega	DevTools, móvil real, `test_ajustes.php`, `clean_test_data.sql`

CAPÍTULO VII

ESPECIFICACIÓN DE LOS SPRINTS

A continuación se describe de forma detallada el trabajo realizado en cada uno de los sprints que conforman la metodología Scrum adaptada al proyecto.

7.1. Sprint 1 – Setup y Autenticación (Semanas 1–2)

El objetivo (*Sprint Goal*) de este sprint fue dejar preparado el entorno de desarrollo y sentar las bases del sistema con el módulo de autenticación.

7.1.1. Actividades realizadas

- Configuración del entorno de desarrollo `DDEV` con PHP 8.4 y MariaDB, inicialización del repositorio `Git`.
- Diseño y creación del esquema relacional de la base de datos junto con los datos semilla (*seeds*).
- Implementación de la autenticación: inicio de sesión mediante cédula o nombre de usuario, contraseña, verificación de cuenta activa y redirección según rol.
- Establecimiento de una estructura base de código ligera que separa la lógica común en el directorio `includes/`.

7.1.2. Producto resultante

Un módulo de autenticación funcional y una base de datos poblada, con la estructura base lista para el desarrollo de los siguientes sprints.

7.2. Sprint 2 – Núcleo QR: Generar y Escanear (Semanas 3–4)

El objetivo de este sprint fue implementar la funcionalidad central del sistema: la generación y el escaneo de códigos QR.

7.2.1. Actividades realizadas

- El profesor genera un código QR único con expiración configurable y rotación automática.
- El estudiante escanea el QR mediante la cámara de su dispositivo (Camera API), se valida el QR y se registra la asistencia mediante peticiones AJAX.

7.2.2. Producto resultante

Un núcleo QR completo que permite generar, escanear y registrar asistencia, constituyendo el corazón funcional del sistema.

7.3. Sprint 3 – Panel Admin: Gestión y Reportes (Semanas 5–6)

El objetivo fue dotar al administrador de las herramientas de gestión y de análisis de la información.

7.3.1. Actividades realizadas

- CRUD de usuarios e importación masiva de inscripciones desde archivos CSV.
- Gestión de cursos e inscripciones.
- Reportes de asistencia con filtros y exportación a CSV.
- Generación de respaldos de la base de datos y registro de auditoría.

7.3.2. Producto resultante

Un panel de administración completo que abarca la gestión de datos, los reportes, los respaldos y la auditoría del sistema.

7.4. Sprint 4 – Pruebas, Validación y Entrega (Semanas 7–8)

El objetivo fue consolidar el sistema mediante pruebas integrales y dejar la solución lista para su entrega.

7.4.1. Actividades realizadas

- Pruebas integrales *cross-módulo* que verifican la interacción entre los distintos componentes.
- Corrección de los errores detectados durante las pruebas.
- Validación móvil: comportamiento responsive y funcionamiento de la cámara en dispositivos reales.
- Limpieza de la base de datos entre ciclos de prueba mediante el script `sql/clean_test_data.sql`.
- Documentación final y despliegue en el entorno `DDEV`.

7.4.2. Producto resultante

Un sistema validado y listo para la entrega, que cumple con los casos de uso definidos.

CAPÍTULO VIII

MODELO DE PRUEBAS DEL PROGRAMA

8.1. Estrategia de Pruebas

El modelo de pruebas adoptado es **manual funcional** por módulo, ejecutado en el entorno `DDEV`, sin frameworks automatizados (no existen en el proyecto). Se utilizan los recursos reales disponibles:

- Script `test_ajustes.php` como patrón para simular sesiones de usuario.
- Script `sql/clean_test_data.sql` para limpiar estado de la base de datos entre ciclos de prueba.
- Navegador web + Chrome DevTools (modo responsive) + dispositivo físico móvil para validar el escaneo de cámara.

La estrategia cubre cuatro niveles:

Unitarias manuales: Funciones de `includes/` (`functions.php`, `academico.php`, `session.php`) invocadas mediante scripts tipo `test_ajustes.php`.

Integración: Flujos completos CU-01 a CU-06 (login →generar QR →escanear →asistencia registrada →reporte).

Funcionales: Cada caso de uso con pasos definidos, entrada y resultado esperado.

Usabilidad móvil: Escaneo QR en dispositivo real, responsive en DevTools.

8.2. Plan por Módulos Reales

Módulo	**Qué se prueba**
Auth (`login.php`, `includes/session.php`)	Login válido/inválido, roles (admin/profesor/estudiante), sesión, logout
Profesor (`generar_qr.php`, `asistencia.php`, `ajustes.php`)	Generar QR, rotación automática, expiración, ver asistencias en tiempo real, configurar parámetros
Estudiante (`escanear_qr.php`, `index.php`)	Escaneo cámara, validación QR (vigente/expirado/inválido), registro asistencia, historial
Admin (`usuarios.php`, `cursos_lista.php`, `reportes.php`, `backup_db.php`, `seguridad.php`)	CRUD usuarios, importar CSV, reportes filtrados, exportar CSV, backup BD, auditoría

8.3. Descripción del Modelo de Pruebas

El modelo de pruebas se documenta mediante casos de prueba representativos que cubren los seis casos de uso del sistema (CU-01 a CU-06). Cada caso de prueba especifica: identificador, módulo, descripción, pasos de ejecución, resultado esperado y tipo de prueba.

ID	Módulo	Descripción	Pasos / Resultado Esperado	Tipo
CP-01	Auth	Login profesor válido	1. Abrir login 2. Credenciales prof 3. Submit →Redirect a `profesor/index.php`	Funcional
CP-02	Auth	Login credenciales inválidas	1. Credenciales erróneas 2. Submit →Mensaje error, permanece en login	Funcional
CP-03	QR	Generar QR profesor	1. Login prof 2. Seleccionar curso/semana 3. Generar →QR visible, expira 5 min, rota 30s	Funcional
CP-04	QR	Escanear QR válido	1. Login est 2. Abrir escáner 3. Apuntar QR →"Asistencia registrada", BD insert	Integración
CP-05	QR	Escanear QR expirado	1. QR con expiración pasada 2. Escanear →Rechaza, notifica "QR expirado"	Funcional
CP-06	Admin	Importar CSV inscripciones	1. Login admin 2. Cargar CSV válido 3. Procesar →Alumnos inscritos en BD	Funcional
CP-07	Admin	Reporte asistencia + CSV	1. Filtros curso/semana 2. Exportar →CSV descargado con datos correctos	Funcional
CP-08	Móvil	Escanear QR en dispositivo real	1. Abrir `escanear_qr.php` en móvil 2. Permitir cámara 3. Apuntar QR →Registro exitoso	Usabilidad

Criterios de aceptación: coinciden con las postcondiciones de los seis casos de uso documentados (CU-01 a CU-06). Una funcionalidad se considera aprobada cuando todos sus casos de prueba asociados pasan sin desviaciones.

CAPÍTULO IX

FACTIBILIDAD ECONÓMICA

El presente proyecto corresponde a un trabajo académico desarrollado por dos estudiantes, por lo que el desarrollo del software no genera un costo de remuneración laboral. Asimismo, el sistema está diseñado para instalarse sobre la infraestructura de equipos y servidores ya existentes en la universidad, empleando únicamente tecnologías de código abierto (PHP, MariaDB, DDEV, HTML, CSS y JavaScript), por lo que no se requiere la adquisición de hardware o licencias de software.

9.1. Costos Humanos

Recurso	**Costo**
Desarrolladores (2 estudiantes del proyecto)	0 $ / 0 Bs
Total	**0 $ / 0 Bs**

9.2. Costos Tecnológicos

Recurso	**Costo**
Equipos de cómputo (ya existentes en la institución)	0 $ / 0 Bs
Software (código abierto, sin licencias)	0 $ / 0 Bs
Servicio de internet (ya disponible en la institución)	0 $ / 0 Bs
Total	**0 $ / 0 Bs**

9.3. Costos Totales del Proyecto

Concepto	**Costo**
Costos humanos	0 $ / 0 Bs
Costos tecnológicos	0 $ / 0 Bs
Total del proyecto	**0 $ / 0 Bs**

Dado que el costo total es nulo, el proyecto no representa una inversión económica para la institución. En adelante se expone el análisis de los ahorros y beneficios que la implementación del sistema genera.

9.4. Ahorros y Beneficios

Al tratarse de un proyecto con costo nulo de desarrollo e infraestructura, la factibilidad económica se justifica por los ahorros y beneficios que el sistema aporta a la gestión de asistencia del IUTECP:

- **Ahorro en insumos de impresión:** el método tradicional exige imprimir una lista o planilla de asistencia para cada clase, lo que implica un gasto recurrente de papel y tinta. Con el sistema, la toma de asistencia es 100 % digital, eliminando por completo este gasto recurrente.
- **Ahorro de tiempo del personal docente:** la marcación manual de cada estudiante se reduce a segundos, liberando tiempo de clase que puede destinarse a la actividad académica.
- **Reducción de pérdida de registros:** al centralizar la información en una base de datos, se elimina el riesgo de extravío o deterioro de las planillas físicas.
- **Menos errores y suplantación:** el registro por QR vigente y único reduce los errores humanos y dificulta la suplantación de identidad.
- **Información oportuna:** los reportes de ausentismo en tiempo real permiten tomar decisiones académicas de forma rápida, reduciendo costos de gestión asociados a la detección tardía de inasistencias.

De esta manera, aunque el valor monetario directo del proyecto es nulo, su impacto se traduce en ahorros sostenidos y en la optimización de los procesos de la institución durante toda su vida útil.

CAPÍTULO X

DISEÑO DE LA BASE DE DATOS

La base de datos del sistema, denominada `qr_iutecp`, fue diseñada bajo el modelo relacional utilizando MariaDB con motor InnoDB. Está compuesta por trece tablas que en conjunto permiten gestionar usuarios, cursos, inscripciones, asistencias, sesiones QR, periodos académicos y auditoría.

10.1. Tablas del Sistema

- `usuarios`: Almacena los datos de todos los usuarios del sistema (administradores, profesores y estudiantes). Incluye campos como cédula (único), nombre de usuario (único), contraseña hasheada, correo electrónico, rol y estado (activo/inactivo).

- `carreras`: Contiene el catálogo de carreras ofrecidas por la institución (Informática, Administración, Contaduría).

- `cursos`: Registra las materias asociadas a cada carrera, con su código, nombre, nivel, unidades crédito y sección.

- `inscripciones`: Relaciona estudiantes con los cursos en los que están inscritos para un periodo académico específico.

- `asistencias`: Registra el estado de asistencia (presente, ausente o justificado) de cada estudiante por semana y fecha, con restricción de unicidad por inscripción-semana-fecha.

- `sesiones_qr`: Almacena los códigos QR generados por los profesores, incluyendo el código único, la fecha de expiración, las coordenadas GPS y el radio permitido.

- `configuracion_profesor`: Guarda la configuración personalizada de cada profesor: tiempo de expiración del QR, intervalo de rotación y radio GPS.
- `periodos`: Define los periodos académicos con sus fechas de inicio y fin, incluyendo soporte para estudiantes de nuevo ingreso.
- `auditoria`: Registra todas las acciones relevantes del sistema con identificador del usuario, tipo de acción, descripción, dirección IP y marca de tiempo.
- `prerrequisitos`: Define las dependencias entre cursos para validar la disponibilidad de materias según el avance del estudiante.

10.2. Esquema Relacional

A continuación se presenta el script de creación de las tablas principales, ejecutado para generar la estructura de la base de datos:

```
CREATE TABLE usuarios (
    id INT AUTO_INCREMENT PRIMARY KEY,
    cedula VARCHAR(20) NOT NULL UNIQUE,
    username VARCHAR(50) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    nombre VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL,
    rol ENUM('admin', 'profesor', 'estudiante') NOT NULL,
    carrera_id INT,
    activo TINYINT(1) DEFAULT 1,
    password_temporal TINYINT(1) DEFAULT 1,
    FOREIGN KEY (carrera_id) REFERENCES carreras(id)
);

CREATE TABLE inscripciones (
    id INT AUTO_INCREMENT PRIMARY KEY,
    usuario_id INT NOT NULL,
    curso_id INT NOT NULL,
```

```
    periodo VARCHAR(10) NOT NULL,
    seccion VARCHAR(10),
    UNIQUE KEY (usuario_id, curso_id, periodo),
    FOREIGN KEY (usuario_id) REFERENCES usuarios(id),
    FOREIGN KEY (curso_id) REFERENCES cursos(id)
);

CREATE TABLE asistencias (
    id INT AUTO_INCREMENT PRIMARY KEY,
    inscripcion_id INT NOT NULL,
    semana INT NOT NULL,
    fecha DATE NOT NULL,
    estado ENUM('presente', 'ausente', 'justificado') DEFAULT 'ausente',
    UNIQUE KEY (inscripcion_id, semana, fecha),
    FOREIGN KEY (inscripcion_id) REFERENCES inscripciones(id)
);

CREATE TABLE sesiones_qr (
    id INT AUTO_INCREMENT PRIMARY KEY,
    profesor_id INT NOT NULL,
    curso_id INT NOT NULL,
    semana INT NOT NULL,
    codigo_qr VARCHAR(64) NOT NULL UNIQUE,
    expira_en DATETIME NOT NULL,
    FOREIGN KEY (profesor_id) REFERENCES usuarios(id),
    FOREIGN KEY (curso_id) REFERENCES cursos(id)
);
```

Listing 10.1: Fragmento del esquema de base de datos (`sql/schema.sql`)

CAPÍTULO XI

CONSTRUCCIÓN DEL SISTEMA

11.1. Herramientas de Programación

Para la construcción del sistema se emplearon las siguientes herramientas:

PHP 8.4: Lenguaje de programación del lado del servidor utilizado para toda la lógica del sistema, con soporte nativo para conexiones seguras a bases de datos mediante PDO.

MariaDB: Sistema de gestión de bases de datos relacional utilizado para almacenar la información del sistema, con motor InnoDB para garantizar la integridad referencial.

HTML5, CSS3 y JavaScript: Tecnologías del lado del cliente para la construcción de las interfaces de usuario, usando JavaScript vanilla para minimizar dependencias externas.

DDEV: Entorno de desarrollo basado en Docker que ejecuta el sistema localmente replicando el entorno de producción.

Visual Studio Code: Editor de código fuente utilizado durante el desarrollo.

Git: Sistema de control de versiones para el seguimiento de cambios.

11.2. Librerías de Programas

El sistema organiza su código en librerías reutilizables ubicadas en el directorio `includes/`, que centralizan la lógica común y evitan la duplicación de código.

- `config.php`: establece la conexión a la base de datos mediante PDO, con manejo de errores en modo excepción y juego de caracteres UTF-8.

- `session.php`: funciones de autenticación y control de acceso (`estaAutenticado()`, `redirigirSegunRol()`, `registrarAuditoria()`).
- `functions.php`: funciones de uso general (`generarPasswordTemporal()`, `limpiarInput()`, `generarUsername()`).
- `academico.php`: lógica académica (`obtenerPeriodoActivo()`, `obtenerSemanaActual()`, `verificarPrerrequisitos()`, `obtenerMateriasDisponibles()`).
- `phpqrcode/phpqrcode.php`: biblioteca *PHP QR Code* para la generación de códigos QR, bajo licencia LGPL 3.

11.3. Fragmento de Codificación

A continuación se presenta un fragmento representativo del código implementado: el endpoint encargado de validar el escaneo del código QR por parte del estudiante, una de las funcionalidades medulares del sistema.

```
<?php
require_once '../includes/config.php';
require_once '../includes/session.php';
require_once '../includes/functions.php';

verificarAutenticacion();
validarRol('estudiante');

$usuario_id = $_SESSION['usuario_id'];
$codigo_qr = limpiarInput($_POST['codigo_qr'] ?? '');

if (empty($codigo_qr)) {
    http_response_code(400);
    echo json_encode(['error' => 'Codigo QR requerido']);
    exit;
}

```

```
try {
    $stmt = $pdo->prepare("
        SELECT sq.id, sq.curso_id, sq.expira_en,
               sq.profesor_id, sq.latitud, sq.longitud,
               sq.radio_permitido, i.id AS inscripcion_id
        FROM sesiones_qr sq
        JOIN inscripciones i ON i.curso_id = sq.curso_id
            AND i.usuario_id = ?
            AND i.periodo = (SELECT codigo FROM periodos
                             WHERE activo = 1 LIMIT 1)
        WHERE sq.codigo_qr = ? AND sq.expira_en > NOW()
        LIMIT 1
    ");
    $stmt->execute([$usuario_id, $codigo_qr]);
    $sesion = $stmt->fetch();

    if (!$sesion) {
        http_response_code(404);
        echo json_encode(['error' => 'QR invalido o expirado']);
        exit;
    }

    $semana = obtenerSemanaActual();
    $stmt = $pdo->prepare("
        INSERT INTO asistencias (inscripcion_id, semana, fecha, estado)
        VALUES (?, ?, CURDATE(), 'presente')
        ON DUPLICATE KEY UPDATE estado = 'presente'
    ");
    $stmt->execute([$sesion['inscripcion_id'], $semana]);

    echo json_encode(['exito' => 'Asistencia registrada correctamente']);

} catch (PDOException $e) {
    http_response_code(500);
```

```
52      echo json_encode(['error' => 'Error del servidor']);
53      registrarAuditoria($pdo, $usuario_id,
54          'Error al escanear QR', $e->getMessage());
55  }
56  ?>
```

Listing 11.1: Fragmento del endpoint de escaneo QR (`estudiante/escanear_qr.php`)

El fragmento anterior muestra la lógica de validación del código QR: verifica que el estudiante esté autenticado, que el QR exista y no haya expirado, que el estudiante se encuentre inscrito en el curso correspondiente, y finalmente registra la asistencia en la base de datos. Se emplean sentencias preparadas PDO para evitar inyección SQL y se utiliza el formato JSON para las respuestas.

CAPÍTULO XII

INTERFACES DEL SISTEMA

El sistema cuenta con interfaces diferenciadas para cada rol de usuario, diseñadas bajo los principios de claridad, eficiencia y adaptabilidad a dispositivos móviles.

12.1. Interfaces del Profesor

- **Generación de QR:** pantalla principal donde el profesor selecciona el curso y la semana, y el sistema genera un código QR único con expiración configurable que se autoregenera automáticamente.
- **Panel de asistencias:** muestra en tiempo real el listado de estudiantes inscritos con su estado de asistencia (presente/ausente).
- **Ajustes de configuración:** permite definir el tiempo de expiración del QR (1–60 minutos) y el intervalo de rotación automática (10–300 segundos).

12.2. Interfaces del Estudiante

- **Pantalla de inicio:** muestra las materias en las que el estudiante se encuentra inscrito, con acceso directo al módulo de escaneo.
- **Escaneo QR:** activa la cámara del dispositivo para capturar el código QR proyectado por el profesor y registra la asistencia automáticamente.
- **Historial de asistencia:** presenta el registro cronológico de asistencias del estudiante.

12.3. Interfaces del Administrador

- **Panel principal:** resumen general con estadísticas de asistencia, usuarios activos y alertas importantes.

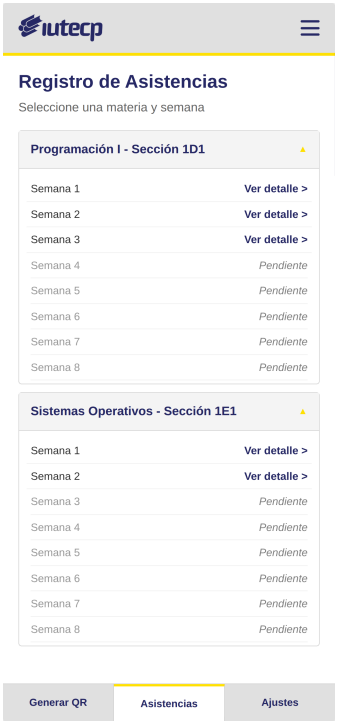
- **Gestión de entidades:** crear, modificar y eliminar usuarios, cursos, carreras y periodos académicos.
- **Reportes y datos:** reportes detallados de asistencia filtrados por curso, semana o estudiante, con opción de exportar a CSV.
- **Seguridad y accesos:** gestión de permisos, roles y auditoría de accesos.

12.4. Galería de Interfaces

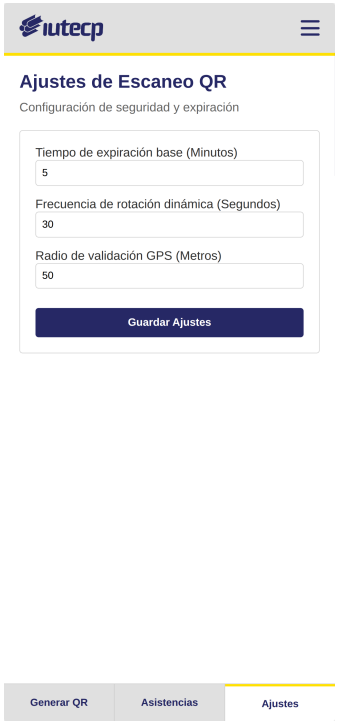
Interfaces del Profesor y Estudiante



Profesor: Generación QR



Profesor: Asistencias



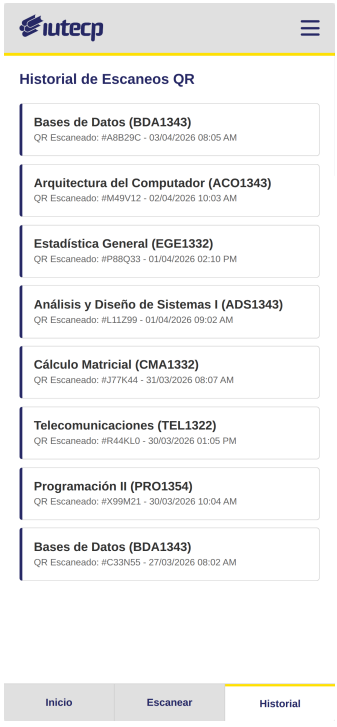
Profesor: Ajustes



Estudiante: Inicio

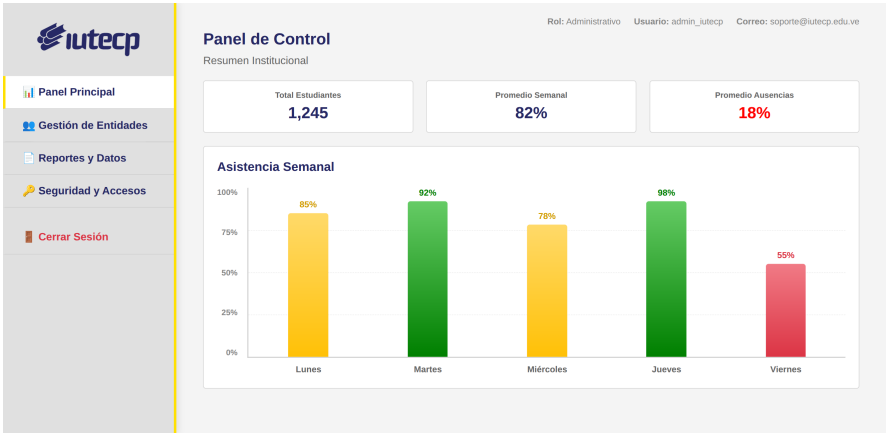


Estudiante: Escaneo QR

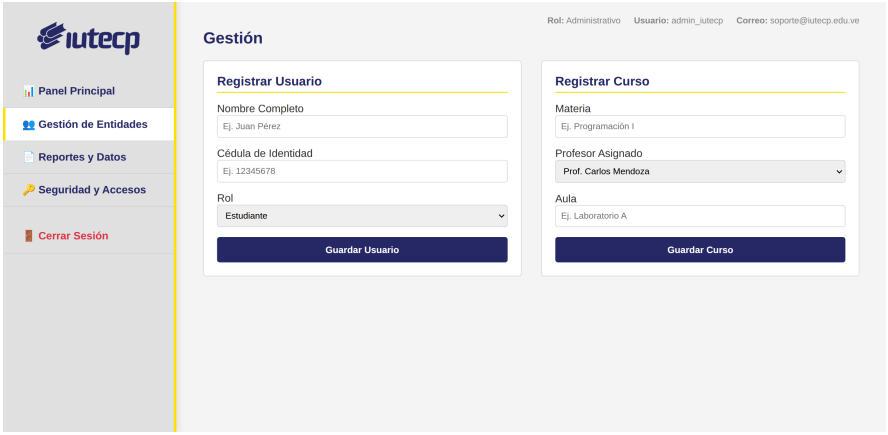


Estudiante: Historial

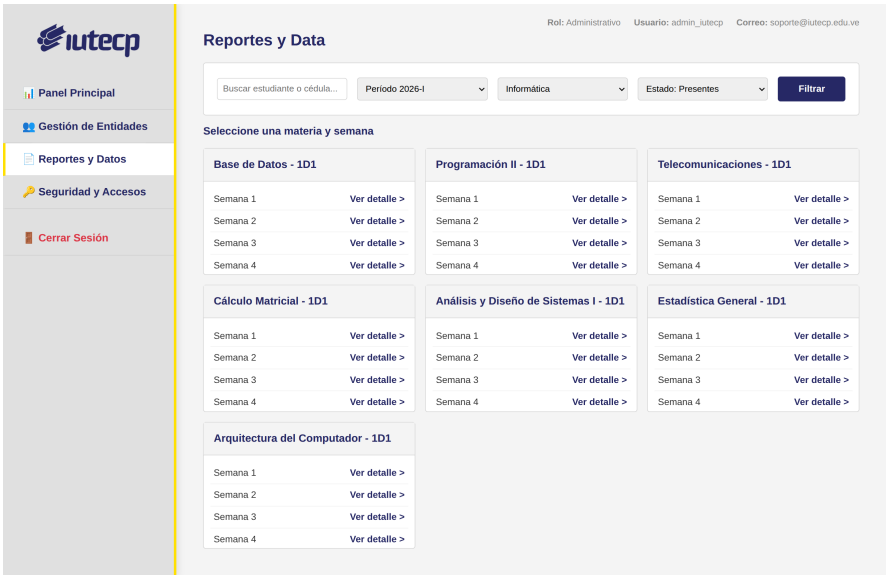
Interfaces del Administrador



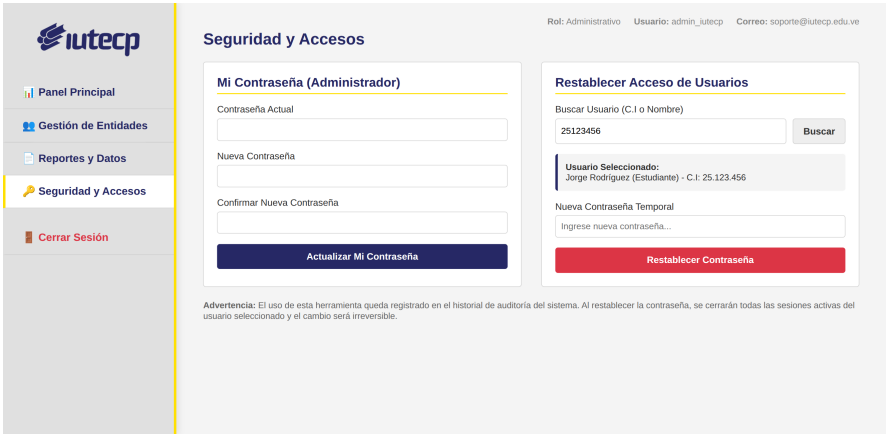
Panel principal



Gestión de entidades



Reportes y datos



Seguridad y accesos

CAPÍTULO XIII

DISEÑO NO FUNCIONAL

El diseño no funcional del sistema aborda las restricciones de calidad que garantizan un funcionamiento seguro, usable y accesible.

13.1. Seguridad y Privacidad

13.1.1. Autenticación y Control de Acceso

- Las contraseñas se almacenan utilizando la función `password_hash` de PHP con el algoritmo `PASSWORD_DEFAULT`.
- En el primer inicio de sesión, se asigna una contraseña temporal que debe ser cambiada obligatoriamente.
- Cada usuario posee un rol (administrador, profesor o estudiante) que determina los módulos y acciones a los que tiene acceso.
- Las cuentas pueden ser desactivadas por el administrador, impidiendo el acceso incluso con credenciales válidas.
- El administrador no puede autodestruirse ni auto-desactivarse, garantizando la integridad del control del sistema.

13.1.2. Protección contra Ataques

- Las consultas a la base de datos utilizan sentencias preparadas (PDO), eliminando el riesgo de inyección SQL.
- Los datos ingresados por los usuarios se sanitizan para neutralizar intentos de inyección de código malicioso (XSS).

- Los códigos QR se generan con entropía criptográfica y expiración configurable, asociándose a una sesión de clase única para impedir su reutilización.

13.1.3. Auditoría

El sistema registra todas las acciones relevantes en una tabla de auditoría que almacena: identificador del usuario, tipo de acción, descripción detallada, dirección IP y marca de tiempo. Esto permite rastrear actividades inusuales y mantener un historial de uso.

13.2. Usabilidad

- Las interfaces presentan una navegación jerárquica adaptada al rol del usuario, con paneles resumidos que permiten acceso rápido a las funcionalidades principales.
- El proceso de escaneo QR se realiza directamente desde la cámara del dispositivo sin instalar aplicaciones de terceros, con retroalimentación visual inmediata.
- Los formularios incluyen validación en tiempo real con mensajes de error específicos, y las acciones destructivas requieren confirmación previa.

13.3. Accesibilidad

- Las interfaces se adaptan a diferentes tamaños de pantalla (responsive) y son funcionales en navegadores modernos sin extensiones adicionales.
- El sistema funciona en condiciones de conectividad variable al minimizar la transferencia de datos y utilizar respuestas ligeras.
- No se requieren dispositivos especializados; cualquier teléfono inteligente con cámara puede utilizarse para escanear el código QR.
- Los elementos interactivos son accesibles mediante teclado, las imágenes incluyen texto alternativo y los contrastes cumplen con pautas de legibilidad.

CONCLUSIÓN

El presente informe final ha documentado de manera integral el desarrollo del Sistema de Gestión de Asistencia mediante Códigos QR para el IUTECP, abarcando los aspectos funcionales, metodológicos, económicos y de calidad de la solución.

El análisis de la problemática actual de control de asistencia manual permitió identificar las deficiencias del método tradicional —pérdida de tiempo, gasto en insumos de impresión, errores humanos y suplantación de identidad— y fundamentar la necesidad de un sistema automatizado. La arquitectura cliente-servidor propuesta, basada en tecnologías web estándar y de código abierto, asegura compatibilidad multiplataforma y una implementación sin costos de licenciamiento.

La metodología Scrum adaptada a un equipo de dos desarrolladores y cuatro sprints permitió un desarrollo incremental y controlado, dejando como resultado la tabla metodológica y la especificación detallada de cada sprint. El modelo de pruebas funcionales por módulo garantizó la validación del sistema de forma manual y fiel al funcionamiento real del proyecto.

La factibilidad económica evidenció que el proyecto no representa una inversión para la institución, al desarrollarse por estudiantes sobre la infraestructura y el software existentes, y que su implementación genera ahorros sostenidos al eliminar la impresión de planillas y optimizar el tiempo de los docentes.

En cuanto al diseño, se establecieron medidas de seguridad robustas (autenticación por roles, sentencias preparadas, protección contra XSS, generación criptográfica de QR y auditoría), así como criterios de usabilidad y accesibilidad que garantizan un sistema práctico y de fácil adopción.

Se concluye que el sistema constituye una solución integral, segura, económicamente viable y eficiente para la gestión de asistencias en el IUTECP.

REFERENCIAS

1. Scrum Guide. (2020). *The Scrum Guide*. ScrumGuides.org.

2. Pressman, R. S. (2014). *Ingeniería del software: Un enfoque práctico* (7.a ed.). McGraw-Hill.

3. Sommerville, I. (2011). *Ingeniería de software* (9.a ed.). Pearson.

4. Booch, G., Rumbaugh, J., & Jacobson, I. (2005). *El lenguaje unificado de modelado: Guía del usuario* (2.a ed.). Addison-Wesley.

5. Object Management Group. (2017). *Unified Modeling Language (UML), versión 2.5.1*. OMG.

6. PHP Group. (2024). *PHP Manual*. `https://www.php.net/manual`

7. MariaDB Foundation. (2024). *MariaDB Knowledge Base*. `https://mariadb.com/kb`

8. Dominguez, D. (2010). *PHP QR Code Encoder*. `http://phpqrcode.sourceforge.net`